

sources, including HVACs, computer systems, inductive loads (motors and pumps) and other lighting systems, which all put out plenty of conducted noise. On top of all this, we also live in a wireless world, which includes mobile phones, Wi-Fi and broadcast radios/televisions.

So, any lighting system that wants to include capacitive touch, and especially proximity interfaces, will require capacitive systems that can operate in the presence of both radiated and conducted noise sources. Fortunately, most capacitive-touch systems currently on the market can tolerate noise levels that are typically encountered in home and office environments. As designers, we just have to make sure we verify that the prefabricated capacitive-touch system is rated for the level of noise we are likely to encounter.

Implementing this into the interface

Now that we have a noise-resistant capacitive touch/proximity system, how can we use it for our interface? Well, in the simplest system, we just have to turn the lighting on and off. However, most higher-end systems are also going to need dimming, so we really need an interface with some level of gesture-recognition. Further, because handing out a users' manual with a light switch is impractical, gestures used will have to be intuitive for the user. Finally, whatever system we use, it has to be reasonably immune to false triggering.

The main requirements are:

1. Simple and intuitive method for turning lights on and off
2. Simple and intuitive method for dimming the lights
3. Reasonable immunity to false triggering
4. Minimal power consumption when not in use
5. Low material cost

From requirements one and two, we know we will probably need some kind of legend explaining the operation of the interface. Given that we are

also talking about lighting control, our legend will either have to glow in the dark or have some kind of tactile information. From requirement three, we know we cannot just toggle the light on and off when a user passes in proximity to the switch.

Requirements four and five indicate that we will have to limit both power and cost. Fortunately, most modern microcontrollers can operate on very little power. These also have all necessary peripheral functions required for both lighting control and capacitive touch.

Therefore, given all these requirements, a reasonable interface would use capacitive proximity to turn on the backlighting for the button legends and a basic buttons-and-slider interface for turning the lights on and off, and dimming. All necessary functions to implement the design should be available in a variety of single-chip microcontrollers, and the necessary capacitive UI software should be readily available from the microcontroller manufacturer.

The proximity sensor on the button legends allows the user to find the switch and turn on the legends, even in the dark. The lighted legends provide him or her with basic instructions for the use of the system. If we also add a software lockout on the button-and-slider controls, such that these cannot be activated within two to three seconds of initial proximity detection, this should prevent the accidental setting change if the user brushes past the interface.

We also minimised power and cost, because we can use a less expensive power supply to only supply the energy for backlighting the legend for the 10-20 seconds the user will need to set the lighting level. Our interface also minimises cost because the capacitive proximity and touch-sensor design can be implemented using a low-cost printed-film sensor.

Now, some may ask, why not make the complete interface capacitive-proximity based? For example:

1. If the user moves his or her

hand from left to right, this could turn the light on, and right to left to turn it off.

2. Moving the hand up/down past the sensor could control dimming of the light.

While this is possible with the technology currently available, the question becomes how does it affect user experience? How will the user find the interface in the dark without potentially turning the lights on at full brightness? What happens if the user brushes past the sensor and turns off the lights? Or, what happens when the family dog brushes its tail past the sensor?

While these may sound somewhat contrived, the designer does have to keep in mind that capacitive touch is susceptible to this kind of environmental noise, and should consider how sensitivity of a capacitive-proximity system can affect operation. It is also possible to handle problems in this kind of system using a more robust gesture-recognition system. However, processing requirements for implementing even a simple two-to-three gesture pattern-recognition system are typically beyond the capability of small, low-cost microcontrollers. So there is a cost trade-off to be considered.

Some things to keep in mind

Capacitive proximity and capacitive touch are exciting new technologies, but the designer must remember that these bring not only new freedom in design but also new challenges that have to be considered when designing a UI. Noise-susceptibility, both electrical and environmental, as well as complexity of the UI experience, has to be considered in design. Remember, it is not just a case of substituting one switch for another. Instead, it is a completely new technology with its own set of advantages and challenges. After all, the novelty of a new interface fades quickly if it is difficult to use and is susceptible to new factors from which the previous system was immune. ●